

Smart Traffic Light Controller

ECE 5367 Term Report

Timothy Truong

Abdur Aziz

Brian Steer

Alaia Homawoo

Ehi Njoku

ECE 5367

University of Houston

Houston, Texas

Abstract – A common problem at four-way traffic intersections is that traffic lights do not operate efficiently during rush hour and later into the night. This results in a power loss, which can be diverted elsewhere. This project aims to replicate a 4 way traffic intersection on a breadboard and operate it using traffic density with a low-power-saving mode feature. The logic was implemented with a software part, written in C++, and a hardware part, built with resistors, LED's, four arduino sensors, and a USB cable to provide power.

I. Introduction

Traffic lights are responsible for the safety of automobile drivers and for the proper flow of vehicles. A traditional traffic light system operates on a constant timing schedule that is based on historical data or simple patterns rather than real-time conditions. However, in car-centric cities, such as Houston, Texas, the traditional traffic light does not adapt to the current traffic density. Additionally, it fails to account for variability. For instance, a traffic light intersection undergoing some sort of construction may not take into account a lack of traffic flow. This would cause delays for the traffic on the other side of the intersection.

This project specifically targets 4-ways intersections with 1 forward lane at each intersection as it is most often neglected to implement technology to control traffic flow.

II. Methodology

A. Specifications

After the project discussion was completed, below were the objectives of the project:

- Construct a density based Arduino Smart Traffic Light System using a circuit breadboard and C++ software code
- Integrate a Low Power Mode feature to reduce the clock speed by half in absence of traffic per each intersection
- Leverage Amdhal's law to account for saved power

Since this project required integrated hardware and software, C++ language code was leveraged since Arduino software code is written in C++. Below were the hardware specifications for this project:

- Four Arduino ultrasonic sensors
- Eight Green LEDs and four Red LEDs
- Eight 220 [Ω] resistors to ground the pins and three 100 [Ω] resistors
- Male to Female Wires
- Solderless breadboard

- Microsoft Windows 11 Laptop & Arduino IDE

B. Defining the Logic

Before writing the C++ code, we came up with a flowchart design that specified the algorithm on which the traffic light signals were going to operate shown in Figure 1 below. It's important to note that this traffic light intersection was built with 4 roadways, as this simulates a real-world intersection, in order to align with the goal of this project.

The benefit of this logic is that the traffic light intersections operate based on traffic density and the delay, measured in seconds, between 2 vehicles that have crossed the intersection, rather than fixed time intervals alone. Additionally, when there is no traffic for more than 10 seconds, low power mode is activated. Our default case is that there is no traffic, so all the red LEDs will be activated when the arduino code is connected to the breadboard via a USB cable. In this project, we define Signal 1, Signal 2, Signal 3, and Signal 4 to be the arduino sensors starting from the rightmost signal and rotating clockwise, respectively. By default, all traffic light signals will be red. Say, for instance, there are two vehicles A and B. Vehicle A arrives at Signal 1, and in two seconds, Vehicle B arrives at Signal 3. This means that traffic signal 1 will turn green for Vehicle A to proceed, and since Vehicle B waits at Signal 3, Signal 1 will wait for 10 seconds, transition to yellow light, and finally to red light. Signal 2 will be skipped since there is currently no traffic at that particular stop.

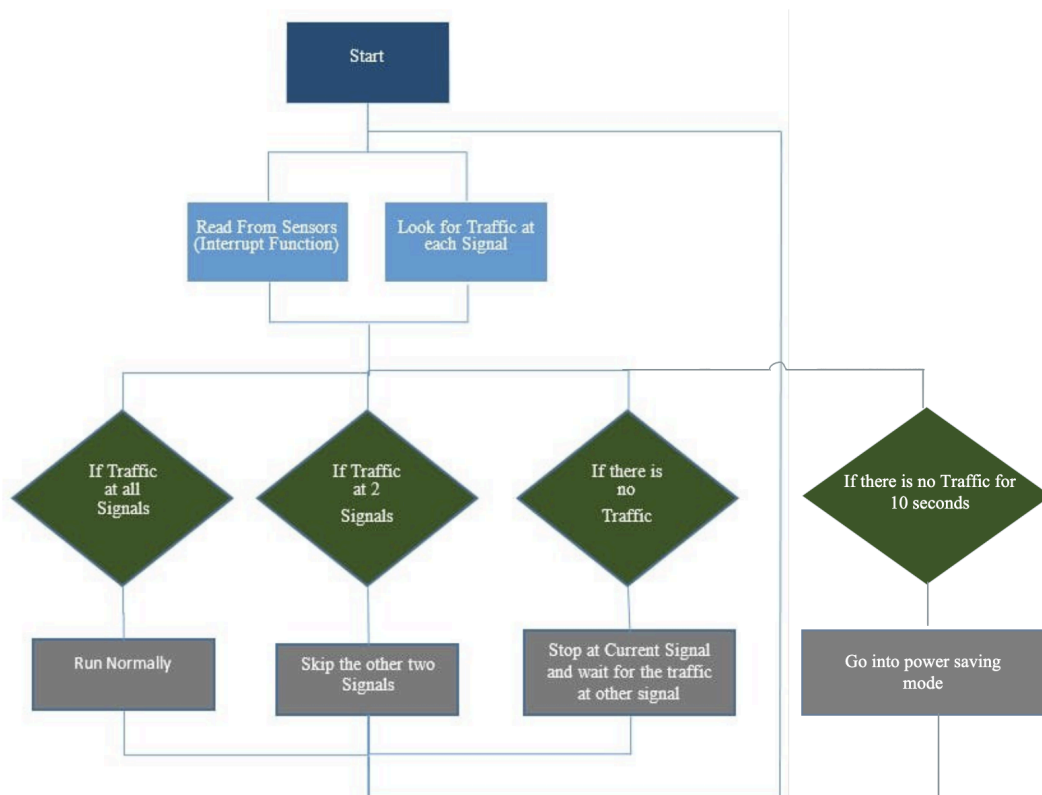


Figure 1. The four traffic light signals operate based on the pattern above.

C. Code

As previously stated, this is an integrated hardware and software project, so the beginning portion of the Arduino code encompasses code that sets up two C++ libraries and defines the connection with the TriggerPins and EchoPins. TimerOne library uses precise timing or periodic interrupt-based tasks are required. Perhaps the most important feature of our project is the LowPower library, which conserves energy so power usage can be optimized for the battery operated sensors. The trigger pin is used to send out a short ultrasonic pulse to initiate the measurement, and the sensor emits an ultrasonic wave and then listens for its reflection. The ultra pin is used to receive the ultrasonic waves that are bounced back from an object after the sensor sends out a pulse. The pin integers represent the pin numbers of the Arduino sensors. All of these features are shown in Figure 2 below.

```
#include<TimerOne.h>
#include <LowPower.h>

int signal1[] = {23, 25, 27};
int signal2[] = {46, 48, 50};
int signal3[] = {13, 12, 11};
int signal4[] = {10, 9, 8};
int redDelay = 5000;
int yellowDelay = 2000;
volatile int triggerpin1 = 31;
volatile int echopin1 = 29;
volatile int triggerpin2 = 44;
volatile int echopin2 = 42;
volatile int triggerpin3 = 7;
volatile int echopin3 = 6;
volatile int triggerpin4 = 5;
volatile int echopin4 = 4;
volatile long time; // Variable for storing the time traveled
volatile int S1, S2, S3, S4; // Variables for storing the distance covered
int t = 5; // distance under which it will look for vehicles.
```

Figure 2. The Arduino code is set up to declare and initialize 4 signal sensor arrays, delay times, and distances, all in type integer.

The code setup continues shown in Figure 3 below. Upon initialization, the Serial communication interface is established at a baud rate of 115200 for debugging and data logging purposes. Subsequently, the Timer1 library is employed to initialize a timer with a specified period of 100,000 microseconds, enabling periodic execution of a user-defined interrupt service routine (ISR). This ISR, denoted as "softInterr", facilitates the execution of tasks at predefined intervals, contributing to efficient event-driven operation. In more details, the softInterr function is called every 100,000 microseconds or 0.1 seconds to calculate the distance measured from the ultrasonic sensors. Below is a code snippet from the softInterr function. It primarily works with the trigger and echo pins of the ultrasonic sensors. To measure the time it takes for the ultrasonic wave to travel to the car and back, the pulseIn function is used. Once, that time is measured, the formula to calculate distance which is $\text{distance} = \text{time} * \text{velocity}$ is used to get the distance in centimeters.

```

void softInterr()
{
  // Reading from first ultrasonic sensor
  digitalWrite(triggerpin1, LOW);
  delayMicroseconds(2);
  digitalWrite(triggerpin1, HIGH);
  delayMicroseconds(10);
  digitalWrite(triggerpin1, LOW);
  time = pulseIn(echopin1, HIGH);
  S1= time*0.034/2;
}

```

Figure 3.

To implement the Low Power Mode feature, a timer must be utilized to count for 10 seconds since the time the system detects no cars. The myMillis structure utilizes the built-in millis() function to add extra features such as resetting the timer, which will be necessary to implement as a timer to count to 10 seconds.

```

//Timer to count seconds
struct myMillis{
  unsigned long offset = 0;
  unsigned long get(){
    return millis() - offset;
  }

  void reset(){
    offset = millis();
  }

  void set(unsigned long value){
    offset = millis() - value;
  }
};

myMillis MM;

```

Figure 4.

The code snippet below for setup() configures the pins corresponding to power-saving indicators and ultrasonic sensors as output and input, respectively. Additionally, clock prescaling (CLKPR) is enabled to optimize clock frequency and minimize power consumption during idle periods. The pinMode() function is used to configure the mode of operation for digital pins on the Arduino board.

```

void setup(){
  Serial.begin(115200);
  Timer1.initialize(100000); //Begin using the timer.
  Timer1.attachInterrupt(softInterr); //Run a function

  // Declaring LED pins as output
  pinMode(powerSavingPin, OUTPUT);
  digitalWrite(powerSavingPin, LOW);
  for(int i=0; i<3; i++){
    pinMode(signal1[i], OUTPUT);
    pinMode(signal2[i], OUTPUT);
    pinMode(signal3[i], OUTPUT);
    pinMode(signal4[i], OUTPUT);
  }
  // Declaring ultrasonic sensor pins as output
  pinMode(triggerpin1, OUTPUT);
  pinMode(echopin1, INPUT);
  pinMode(triggerpin2, OUTPUT);
  pinMode(echopin2, INPUT);
  pinMode(triggerpin3, OUTPUT);
  pinMode(echopin3, INPUT);
  pinMode(triggerpin4, OUTPUT);
  pinMode(echopin4, INPUT);

  CLKPR = 0x80; // Enable clock prescale
  CLKPR = 0x00; // Use clock division factor 1
}

```

Figure 5. The Arduino Code setup is continued here from Figure 1, displaying the setup of connections for the Echo Pins and Trigger Pins.

The loop() function serves as the central control mechanism for an Arduino-based traffic light system. It orchestrates the operation of the traffic lights in response to vehicle presence detected by sensors at each intersection. What the loop function does is to continuously check for distance at each intersection to see if the distance is less than 5 (cm) as seen in the 4 if statements in the code snippet below. There is also another if-else statement that covers the 4 if statements. The if-else statement is for Low Power Mode. The if part is to check if there is no vehicle detected, then activates a timer, and the else part is for normal operation when cars are detected or no cars detected for less than 10 seconds.

Upon initialization, the function declares a variable to track the current time. It then implements a power-saving mode, activated when no vehicles are detected at any intersection for a predefined period. During power-saving mode, the system reduces its clock speed to conserve energy, indicated by toggling an LED connected to a designated pin. Upon detection of vehicle activity, the system exits power-saving mode, restores the original clock speed, and resumes normal operation. This includes controlling the traffic lights at each intersection based on vehicle presence, facilitated by dedicated signal functions. Serial output is utilized for debugging and monitoring, providing real-time information about system status, including current time, power-saving mode activation, and flag statuses. Through dynamic adjustment of traffic light behavior and energy-efficient operation, the loop() function ensures efficient management of the

traffic light system, enhancing both functionality and power. conservation.

```
void loop()
{
    long currentMillis;
    // If there are vehicles at signal 1
    if ((S1 >= t) && (S2 >= t) && (S3 >= t) && (S4 >= t)) {
        digitalWrite(powerSavingPin, HIGH);
        currentMillis = MM.get(); //start timer
        //If more than 10s without traffic, activate power saving mode
        if (currentMillis >= waitTime){
            if (flag1 == 0 && flag2 == 1){
                lowPowerSetup(); // Use clock division factor 2
                flag1 = 1;
                flag2 = 0;
                Serial.println("Low");
                flag0 = 1;
            }
            Serial.println(currentMillis);
            digitalWrite(powerSavingPin, HIGH);
            delay(3000);
            digitalWrite(powerSavingPin, LOW);
            delay(3000);
            Serial.println(flag1);
            Serial.println(flag2);
        }
    }
    else {
        if (flag1 == 1 && flag2 == 0){
            setup();// Go back to original clock speed / get out of power saving mode
            flag1 = 0;
            flag2 = 1;
            digitalWrite(powerSavingPin, LOW);
        }
        // If there are vehicles at signal 1
        if(S1<t)
        {
            signal1Function();
        }
        // If there are vehicles at signal 2
        if(S2<t)
        {
            signal2Function();
        }
        // If there are vehicles at signal 3
        if(S3<t)
        {
            signal3Function();
        }
        // If there are vehicles at signal 4
        if(S4<t)
        {
            signal4Function();
        }
    }
}
```

Figure 6. loop() Function

Figure 4. The flag1 and flag2 functions above manage the traffic light's systems between power-saving mode and normal operation mode.

The provided code snippet encapsulates a function named `low()`, which forms an integral component of the Arduino traffic light system. This function is devised to regulate the behavior of the traffic lights, specifically ensuring that all LEDs, with the exception of the red ones, are deactivated. Executed within the context of nested for loops, the function systematically traverses through the LED pins associated with each traffic light signal. Within the inner loop, a series of `digitalWrite()` commands are employed to set the state of these pins to LOW, effectively extinguishing all LEDs connected to them. Subsequently, the function proceeds to illuminate the red LEDs by setting the state of the first LED pin (index 0) for each traffic light signal to HIGH. This operation, in essence, establishes the traffic lights in a configuration where only the red light is illuminated, signifying the cessation of vehicular movement at all intersections. Such a state aligns with the standard traffic light cycle, where the activation of the red light denotes the requirement for vehicles to come to a halt. Consequently, the `low()` function plays a pivotal role in orchestrating the traffic light system, facilitating the efficient management of vehicular traffic flow and ensuring adherence to established traffic regulations.

```
// Function to make all LED's LOW except RED one's.
void low()
{
    for(int i=1; i<3; i++)
    {
        digitalWrite(signal1[i], LOW);
        digitalWrite(signal2[i], LOW);
        digitalWrite(signal3[i], LOW);
        digitalWrite(signal4[i], LOW);
    }
    for(int i=0; i<1; i++)
    {
        digitalWrite(signal1[i], HIGH);
        digitalWrite(signal2[i], HIGH);
        digitalWrite(signal3[i], HIGH);
        digitalWrite(signal4[i], HIGH);
    }
}
```

Figure 7. low() Function

D. Hardware

Now that the software code compiled with no bugs, the hardware part needed to be built. The hardware includes all 4 ultrasonic sensors and 4 sets of red, yellow and green LED, that are all connected and powered from the Arduino Mega 2560 that is powered by a laptop. A battery pack can also be used to power the entire system without a laptop.

III. Conclusion

Our traffic signal system has been successfully completed and is now operational. Utilizing advanced sensors, the system detects approaching vehicles and dynamically adjusts the signal lights accordingly. The sequence begins with a green light lasting for 5 seconds, followed by a 2-second yellow phase, and then transitions to red. One of the innovative features of our system is its adaptive behavior based on traffic conditions. If there are no vehicles approaching from a specific direction, the signal will not change to green for that direction, ensuring efficient traffic flow. Conversely, when multiple vehicles are detected from different directions, the system intelligently adjusts the signal to allow for each vehicle to pass through safely. Additionally, to conserve power, the system automatically enters a power-saving mode if there are no vehicles detected at the intersection for more than ten seconds. In this mode, the system clock speed is reduced by half, optimizing energy usage without compromising functionality.

IV. Prospective Plans

A way to expand on our project would be to integrate the detection and prioritization of pedestrians and cyclists to improve safety and promote alternative modes of transportation. Furthermore, employing an AI-driven system leveraging the ultrasonic sensors could detect traffic congestions, adjusting green light durations for optimized traffic flow in each direction. To align with the idea of power optimisation we could add solar panels and small batteries to power all the signals in the traffic intersection. This would make sense since traffic intersections are usually less active at night, where there is less light and the power saving mode would be mostly on.

V. Reference

- [1]“Density Based Traffic Light Controller Using Arduino,” *Hackster.io*.
<https://www.hackster.io/muhammad-aqib/density-based-traffic-light-controller-using-arduino-8636ad#comments> (accessed Apr. 29, 2024).
- [2]“How to optimize Arduino for low power design,” *www.microcontrollertips.com*.
<https://www.microcontrollertips.com/how-to-optimize-arduino-for-low-power-design/#:~:text=T>

o%20activate%20idle%20mode%20in%20ATmega2560-based%20Arduino%2C%20add
(accessed Apr. 29, 2024).